ClariDx

Explainable Clinical Decision Support System — Project Presentation & Viva Guide

Target Audience: Final-Year CS/IS Students, Academic Evaluators, & Healthcare Clients • **Repo:** https://github.com/Vishwas7975/claridx • **Branch:** main

1. Quick Reference: Demo Login Credentials

Use these pre-configured accounts in live evaluations to demonstrate Role-Based Access Control (RBAC):

Role	Email	Password	Allowed Capabilities
Doctor	`doctor@claridx.local`	`ClariDxDemo2026!`	**Full Clinical Dashboard:** Run Heart & Diabetes predictions, view SHAP & LIME attributions, test missing-data sensitivity, load patient presets.
Admin	`admin@claridx.local`	`ClariDxDemo2026!`	**Full Clinical + System Authority:** Full prediction access, system configuration, audit logs, and account management.
Researcher	`researcher@claridx.local`	`ClariDxDemo2026!`	**Read-Only Inspection:** Can query feature definitions and model specs via Swagger UI; **strictly blocked** from running predictions on patient data (HTTP 403 Forbidden).

2. Executive Summary: What Problem Does ClariDx Solve?

The Real-World Barrier: Standard machine learning models achieve high numerical accuracy, but doctors **refuse to trust them** because of the 'Black Box Problem'. An algorithm outputting '85% Risk' without biological explanation cannot be legally or ethically used for patient treatment.

The ClariDx Solution: An Explainable Clinical Decision Support System (CDSS) for Heart Disease and Type 2 Diabetes:

- **Dual Explainable AI:** Computes SHAP and LIME in parallel to explain individual patient risk drivers.
- **Cross-Explainer Agreement:** Computes Spearman correlation and sign consistency so clinicians know when explanations are trustworthy.
- **Missing-Data Sensitivity:** Simulates 100+ empirical background scenarios for unperformed lab tests, displaying risk bands and flagging if a missing test could alter the diagnosis (can_flip).
- **Zero-as-Missing Reality:** Treats impossible physiological zeros (glucose=0, BP=0) as missing data to impute, while preserving valid zeros (pregnancies=0).

3. Technology Stack & Architectural Rationale

Tier	Technology Used	Engineering Justification (Why It Was Chosen)
Frontend Core	React 19 + TypeScript	Compile-time type safety for medical forms, modular components, sub-second reactivity.
Build Tool	Vite	Sub-second HMR and optimized ES-module bundling (builds production bundle in ~460ms).
Server State	TanStack React Query v5	Declarative server-state caching, automatic deduplication, and error boundaries.
Design System	CSS Custom Properties	Zero-dependency styling enforcing strict WCAG AA clinical contrast and responsive layouts.
Backend API	FastAPI (Python 3.12)	Async Python core, native OpenAPI 3.1 schema generation, and automatic Swagger UI.
Validation	Pydantic v2	Rust-powered validation core rejecting invalid physiological data before reaching ML pipelines.
Database & ORM	MySQL 8.0 + SQLAlchemy 2.0	ACID-compliant storage for users and sessions; migrations managed via Alembic.
Machine Learning	Scikit-learn Pipelines	Imputation, scaling, and models encapsulated strictly to **guarantee zero data leakage**.
Explainable AI	SHAP & LIME	Combines game-theoretic Shapley attributions with local surrogate linear models.
Security	Argon2id + PyJWT	PHC winner (memory-hard, GPU-resistant); dummy verification prevents timing enumeration.
Package Tooling	uv (Astral)	Rust-based package manager (10–100x faster than pip) ensuring reproducible lockfiles.

4. Deep-Dive: Heart Disease & Diabetes Workflows

4.1 Heart Disease Risk Prediction (UCI Cleveland Dataset):

Includes 303 patient records with 13 features: age, sex, chest pain type, resting BP, cholesterol, fasting sugar, resting ECG, maximum heart rate, exercise angina, ST depression, slope, vessels colored, and thallium stress test. Minimum 7 of 13 features required. **Selected Model:** Random Forest (ROC-AUC **0.898** on repeated 5×5 stratified cross-validation).

4.2 Type 2 Diabetes Risk Prediction (Pima Indians Dataset):

Includes 768 female patients aged ≥ 21 across 8 features: pregnancies, glucose, blood pressure, skinfold thickness, insulin, BMI, pedigree function, age. Minimum 5 of 8 features required. **Selected Model:** Logistic Regression (ROC-AUC **0.842**).

The Zero-as-Missing Nuance: In physiological tests, glucose=0 or blood pressure=0 is biological death. ClariDx automatically converts impossible zeros in glucose, BP, skinfold, insulin, and BMI to NaN and applies median imputation, while preserving pregnancies=0 as a valid measurement.

5. Explainability & Missing-Data Sensitivity Analysis

Dual XAI: SHAP provides game-theoretic consistency across features, while LIME provides intuitive local linear approximations. Running both in parallel and computing Spearman rank correlation ensures doctors do not rely on a single algorithm's potential artifact.

Sensitivity Simulation: When optional measurements are missing, ClariDx samples 100+ empirical background scenarios from the training cohort. It computes minimum and maximum risk bounds and alerts clinicians if an unperformed lab test has the power to change the diagnosis (can_flip).

6. Step-by-Step Live Demonstration Script

Step 1: Sign-In & Access Policy

Open http://localhost:5173. Point out the split-panel design and click 'Request Access'. Explain that in medical software, self-registration is disabled for clinical governance; accounts are provisioned via admin CLI.

Step 2: Heart Disease & Dual XAI

Sign in as Doctor (doctor@claridx.local / ClariDxDemo2026!). Under Heart Disease, click preset 'Example: typical high risk'. Click 'Analyse'. Show the probability bar (50% threshold), SHAP feature contributions (risk-increasing in red, protective in green), LIME bars, and the Agreement Card.

Step 3: Missing-Data Simulation

Clear the 'Maximum Heart Rate' input and click 'Analyse'. Show the Sensitivity Analysis panel that dynamically appears with the risk range and 'can alter verdict' indicator.

Step 4: Diabetes & Zero-as-Missing

Switch to Diabetes tab. Click preset 'Example: routine clinic (skin & insulin missing)'. Intentionally enter '0' in Plasma Glucose and click 'Analyse'. Point out the warning banner showing that glucose=0 was caught and safely imputed.

Step 5: Role-Based Governance

Log out and sign in as Researcher (researcher@claridx.local / ClariDxDemo2026!). Show that predictions are replaced with a 'Researcher Access (Read-Only)' card, proving strict clinical RBAC.

7. Top 15 Viva & Examination Questions with Answers

1. Why didn't you use Deep Learning (Neural Networks / CNNs)?

Tabular clinical datasets (303 rows for Heart, 768 rows for Diabetes) are small. Deep neural networks easily overfit on small tabular data and act as opaque black boxes. Tree ensembles (Random Forest, Gradient Boosting) consistently achieve higher ROC-AUC on tabular clinical benchmarks while enabling exact game-theoretic SHAP explainability.

2. How did you prevent Data Leakage between train and test sets?

All imputation medians, standard scaling, and categorical encoders were encapsulated inside scikit-learn Pipeline objects. In both cross-validation and holdout evaluation, transformation parameters were learned strictly on training folds and applied statelessly to validation folds. The holdout test set was held in reserve and evaluated only once.

3. What is the 'Zero-as-Missing' problem in the Diabetes dataset?

In historical diabetes records, unrecorded lab measurements were entered as '0'. Since blood glucose, blood pressure, or BMI cannot be zero in a living human, treating them as true zero coordinates corrupts model training. ClariDx distinguishes valid zeros (pregnancies=0) from invalid zeros (glucose=0), converting the latter to NaN before median imputation.

4. Why are model scores labeled as 'Uncalibrated Scores'?

Standard scikit-learn models output raw voting fractions or sigmoid scores that do not represent true statistical Bayesian event probabilities without isotonic or Platt calibration. To maintain clinical honesty, ClariDx explicitly flags all outputs as uncalibrated scores ('calibrated': false), reminding clinicians to treat them as decision scores rather than true clinical risk measures.

5. Why choose Argon2id over bcrypt or SHA-256 for password hashing?

SHA-256 is an unsalted fast cryptographic hash that can be brute-forced at billions of attempts per second using consumer GPUs. Argon2id is the winner of the Password Hashing Competition (PHC); it is both memory-hard and time-hard, resisting GPU, ASIC, and side-channel cache attacks.

6. How does your application protect against User Enumeration via timing attacks?

If an attacker submits an unregistered email on the login page, the backend executes an Argon2id verification cycle against a simulated dummy hash before answering with HTTP 401. This guarantees that response latency is identical whether an email exists in the database or not.

7. What is the purpose of Missing-Data Sensitivity Analysis?

Single imputation assumes a single fixed average value for missing lab tests, hiding clinical uncertainty. ClariDx tests 100+ empirical background scenarios for omitted features to determine minimum and maximum risk bounds and identify whether an unmeasured lab test has the power to change the diagnosis (can_flip).

8. How is end-to-end type safety maintained across Python and TypeScript?

Via our automated 'npm run api:sync' workflow. FastAPI serializes its OpenAPI 3.1 schema to JSON, and openapi-typescript compiles it into static TypeScript definitions (schema.d.ts). Any mismatch between backend response fields and frontend UI triggers an instant compile error at build time.

9. Why did you use 'uv' instead of standard 'pip'?

uv is an ultra-fast Python package manager written in Rust. It resolves and installs packages 10–100× faster than pip, enforces deterministic lockfiles, and ensures identical virtual environment reproducibility between developer and evaluator machines.

10. How was the decision threshold (0.50) chosen?

A 0.50 threshold represents the standard mathematical decision boundary where probability of disease presence exceeds absence. The frontend dynamically reads and renders whatever threshold the API response returns, allowing institutional policy customization.

11. Why is self-registration disabled in the application?

In a hospital or clinical environment, access to patient prediction systems must be strictly governed. Allowing anyone on the internet to self-register as a 'Doctor' violates HIPAA and clinical governance. All user provisioning is restricted to the administrator command-line tool (python -m app.cli.create_user).

12. What role-based restrictions exist for Researchers?

Researchers have read-only access. They can query /api/v1/heart/features, /api/v1/diabetes/features, and model metadata to study benchmark metrics. However, any POST request to prediction endpoints immediately returns HTTP 403 Forbidden with an explanatory UI notice.

13. How were your evaluation metrics chosen?

In medical screening, accuracy alone is misleading due to class imbalances. We prioritized ROC-AUC (discriminative ability across all thresholds), Recall / Sensitivity (minimizing false negatives so sick patients aren't missed), and Brier Score (evaluating probability score calibration).

14. How are automated tests organized and verified?

The project contains 469 automated tests: 278 backend tests in pytest covering API routes, RBAC permission gates, JWT edge cases, pipelines, SHAP/LIME generators, and database migrations; and 191 frontend tests in vitest covering component rendering, form validation, and mock API integration.

15. What are the main clinical limitations of this project?

1. Datasets are historical benchmark cohorts (Cleveland heart study with 303 patients; Pima Indians study limited to Arizona Pima females >= 21), so models cannot be generalized to broader global demographic populations without retraining on local clinical cohorts. 2. Explanations reflect model reasoning, not biological causality. 3. Models must undergo prospective clinical trials and probability calibration before real-world patient deployment.